



INDIA.RUNS

Build what next India runs on

Team Name : RankSense

Team Leader Name : Magi Vaishnavi S

Problem Statement : Intelligent Candidate Discovery & Ranking

Solution Overview

- What is your proposed solution?

- RankSense: a hybrid CPU-only ranker that scores all 100,000 candidates against the JD using 4 signal layers (structured role/career fit, trust-adjusted skill match, TF-IDF semantic similarity, behavioral availability) plus a hard honeypot/disqualifier filter, then outputs a ranked top-100 with grounded, per-candidate reasoning -- full run completes in under 1 minute.

- What differentiates your approach from traditional candidate matching systems?

- No keyword counting and no per-candidate LLM calls. Skill claims are cross-checked against the candidate's own Redrob assessment scores and skill duration -- so a stuffed 'expert' skill with 0 months of use is actively penalized, not rewarded. Disqualifiers (consulting-only, research-only, shallow LangChain-only, stale coders) act as multipliers that a strong skill list cannot simply outvote.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?
 - Production embeddings/retrieval experience, vector DB or hybrid search ops experience, strong Python, and hands-on ranking-evaluation experience (NDCG/MRR/MAP/A-B testing) are treated as core requirements; LoRA/QLoRA, learning-to-rank, HR-tech background, and OSS contributions are treated as bonus, never disqualifying.
- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?
 - Title + career-history role signal (are they actually an ML/search/retrieval practitioner, or just a non-practitioner with AI keywords listed), trust-adjusted skill match against Redrob's own skill_assessment_scores, and TF-IDF semantic overlap between JD text and the candidate's free-text career descriptions -- catching candidates who built the right systems without using the JD's exact vocabulary.

Ranking Methodology

- How does your system retrieve, score, and rank candidates?
 - Every candidate is scored against the single released JD in one batch pass: honeypot filter first, then TF-IDF cosine similarity (corpus-fit, no pretrained weights) for semantic fit, then structured feature scoring, combined into a composite, then re-scaled by hard disqualifier multipliers and the behavioral availability multiplier. Candidates are sorted descending and the top 100 are assigned ranks 1-100.
- What models, algorithms, or heuristics are used?
 - TF-IDF vectorization + cosine similarity (scikit-learn, fit fresh on the candidate corpus each run -- no network, no GPU) for semantic fit; weighted rule-based scoring for role/skill/experience/location/education fit; explicit heuristic disqualifier rules encoding the JD's own stated exclusion list; a behavioral multiplier built from recruiter response rate, last-active recency, interview completion rate, and notice period.
- How are multiple candidate signals combined into a final ranking?
 - $\text{composite} = (0.6 \times \text{structured_fit} + 0.4 \times \text{semantic_fit}) \times \text{disqualifier_multiplier} \times \text{availability_multiplier}$. Disqualifiers and availability are multiplicative, not additive -- one serious red flag (e.g. consulting-only career) meaningfully discounts even a strong skill-list candidate.

Explainability & Data Validation

- How are ranking decisions explained?
 - Each top-100 row gets a reasoning string built directly from that candidate's own real fields (title, company, years, matched core-skill count, response rate, last-active date, and any disqualifier flag that applied) -- never generated by an LLM, so it can never invent a skill or fact not present in the candidate's profile.
- How do you prevent hallucinations or unsupported justifications?
 - Reasoning text is template-assembled from verified fields only (no free-generation step), and phrasing rotates across several wordings keyed off each candidate_id so the 100 reasoning strings are all unique rather than copy-pasted -- verified: 100/100 unique strings in our test run.
- How does your solution handle inconsistent, low-quality, or suspicious profiles?
 - An upstream honeypot/integrity filter checks for internal contradictions: 'expert' proficiency claimed with 0 months of skill use, overlapping current full-time jobs, education end-year before start-year, and future-dated career start dates. Flagged profiles are excluded before scoring. On the full 100K pool this removed 48 candidates; 0 honeypots appeared in our final top-100.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?
 - 1) Load candidates.jsonl (streamed, one JSON object per line). 2) Run honeypot/integrity filter. 3) Fit TF-IDF over the surviving corpus + JD text, compute cosine similarity per candidate. 4) Compute structured role/skill/experience/location/education features plus disqualifier flags per candidate. 5) Combine into a composite score and apply the behavioral availability multiplier. 6) Sort, take top 100, assign ranks with deterministic tie-breaking by candidate_id. 7) Generate grounded reasoning text per row. 8) Write submission.csv and submission.xlsx.

System Architecture

- 1. Ingest -- stream-parse candidates.jsonl (100K records), no full in-memory dataframe required.**
- 2. Integrity Filter -- internal-consistency / honeypot checks exclude impossible profiles before scoring.**
- 3. Semantic Layer -- TF-IDF fit on JD + candidate corpus, cosine similarity per candidate (local, offline, no GPU).**
- 4. Structured Layer -- role fit, trust-adjusted skill match, experience, location, education scored per candidate.**
- 5. Composite + Multipliers -- disqualifier and behavioral-availability multipliers applied on top of the blended score.**
- 6. Output -- top 100 sorted, ranked, deterministic tie-break, reasoning generated,**

Results & Performance

- What results or insights demonstrate ranking quality?
 - Top-ranked candidates in our run are consistently Senior/Staff/Lead AI, ML, NLP, and Search/Recommendation engineers at recognizable product companies (Meta, Flipkart, Salesforce, Microsoft, Netflix, CRED) with multiple JD-core skills backed by Redrob assessment scores, high recruiter response rates, and recent platform activity -- exactly the profile the JD describes as its 'ideal candidate'. Zero honeypots surfaced in the top 100.
- How does your solution meet the challenge's runtime and compute constraints?
 - Measured end-to-end runtime on the full 100,000-candidate pool: ~35-55 seconds, CPU-only, no GPU, no network calls during ranking, well inside the 5-minute / 16GB budget. No pre-computation step is required -- a single command (`python src/rank.py`) produces the final submission CSV directly.

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?
 - Python 3.11, scikit-learn (TfidfVectorizer + cosine_similarity) for fully local, no-download semantic matching, numpy for vectorized scoring, openpyxl for XLSX output, and the standard library csv/json modules for I/O. No hosted LLM APIs, no GPU libraries, and no pretrained-weight downloads are used anywhere in the ranking path -- chosen specifically to satisfy the offline, CPU-only, 5-minute compute constraint while still delivering a genuinely hybrid (rule-based + semantic) ranker.

Submission Assets

GitHub Repository: github.com/Magi0721/ranksense-redrob (public, includes README, src/rank.py, src/scoring.py, requirements.txt, submission_metadata.yaml)

- Sandbox (Colab): hosted notebook (sandbox_colab.ipynb in the repo) that runs src/rank.py end-to-end on a small candidate sample within the compute budget.
- Submission files: ranked top-100 output provided as both submission.csv (per spec Section 2-3) and submission.xlsx (per portal upload requirement).



INDIA.RUNS

Build what next India runs on

THANK YOU